

LOSTFOUND

INTRODUCTION:

LostFound+ is a centralized web-based Lost and Found Management Platform designed to simplify the process of reporting, searching, claiming, and recovering lost belongings. In many colleges, organizations, offices, and public environments, lost items are commonly reported through notice boards, WhatsApp groups, social media platforms, or informal communication. These methods make it difficult to maintain proper records, search for previously reported items, and verify the actual ownership of an item.

LostFound+ provides a single digital platform where users can register securely, report lost or found belongings, search existing records, view detailed item information, and submit ownership claims. Each item can be recorded with important information such as item name, category, description, location, date, and image. This structured information makes it easier for users to identify potentially matching items.

The platform also provides an administrative interface for monitoring reported items and ownership claims. Administrators can review submitted reports, verify claims, approve or reject requests, remove duplicate or inappropriate listings, and maintain the overall quality of the system.

The application follows a simple workflow:

Register/Login → Report Item → Search Items → View Details →
Submit Claim → Admin Verification → Recovery

LostFound+ uses the **MERN stack**, consisting of MongoDB, Express.js, React.js, and Node.js. The combination provides a responsive frontend, RESTful backend services, database management.

SCENARIO BASED CASE STUDY:

Consider a college student named **Arun** who accidentally loses his wallet inside the college campus. In a traditional system, Arun may have to check notice boards, ask security personnel, contact friends, or post messages in different WhatsApp groups.

Arun opens **LostFound+** and creates an account using his basic details. After successful registration, he securely logs into the platform.

Reporting a Lost Item

Arun selects the **Report Lost Item** option and enters the required information:

- Item Name: Wallet
- Category: Personal Belonging
- Description: Black leather wallet
- Lost Location: College Library
- Lost Date: Current Date
- Image: Wallet photograph

The information is submitted and stored in the system.

Searching for the Item

Meanwhile, another student named **Kavin** finds a black wallet near the college library. Instead of using informal communication, Kavin logs into LostFound+ and reports the item as a **Found Item**.

He provides the item name, category, description, found location, date, and image.

Arun searches the platform using keywords and category filters. He finds a found-item listing that appears to match his lost wallet. He opens the item details page and reviews the available information.

SOFTWARE REQUIREMENTS:

The LostFound+ web application requires a suitable software environment for development, testing, database management, and deployment.

1. Operating System

Windows 10/11, macOS, or Linux can be used for development and testing. The MERN stack supports cross-platform development.

2. Node.js

Node.js provides the runtime environment required for executing the backend application. It is used to run the Express.js server and handle API requests. 🖱️ Download Node.js

3. npm

npm is used as the package manager for installing and maintaining project dependencies required by both the frontend and backend.

4. React.js

React.js is used to develop the responsive and interactive frontend interface. Components can be reused across different pages of the application.

5. Express.js

Express.js is used to create RESTful APIs and handle backend routing, authentication, CRUD operations, validation, and business logic.

6. MongoDB

MongoDB is used as the primary database for storing users, lost items, found items, claims, and administrative information.
<https://www.mongodb.com/try/download/community>

7. MongoDB Compass

MongoDB Compass can be used to view collections, inspect stored documents, and manage the database during development.

8. Visual Studio Code

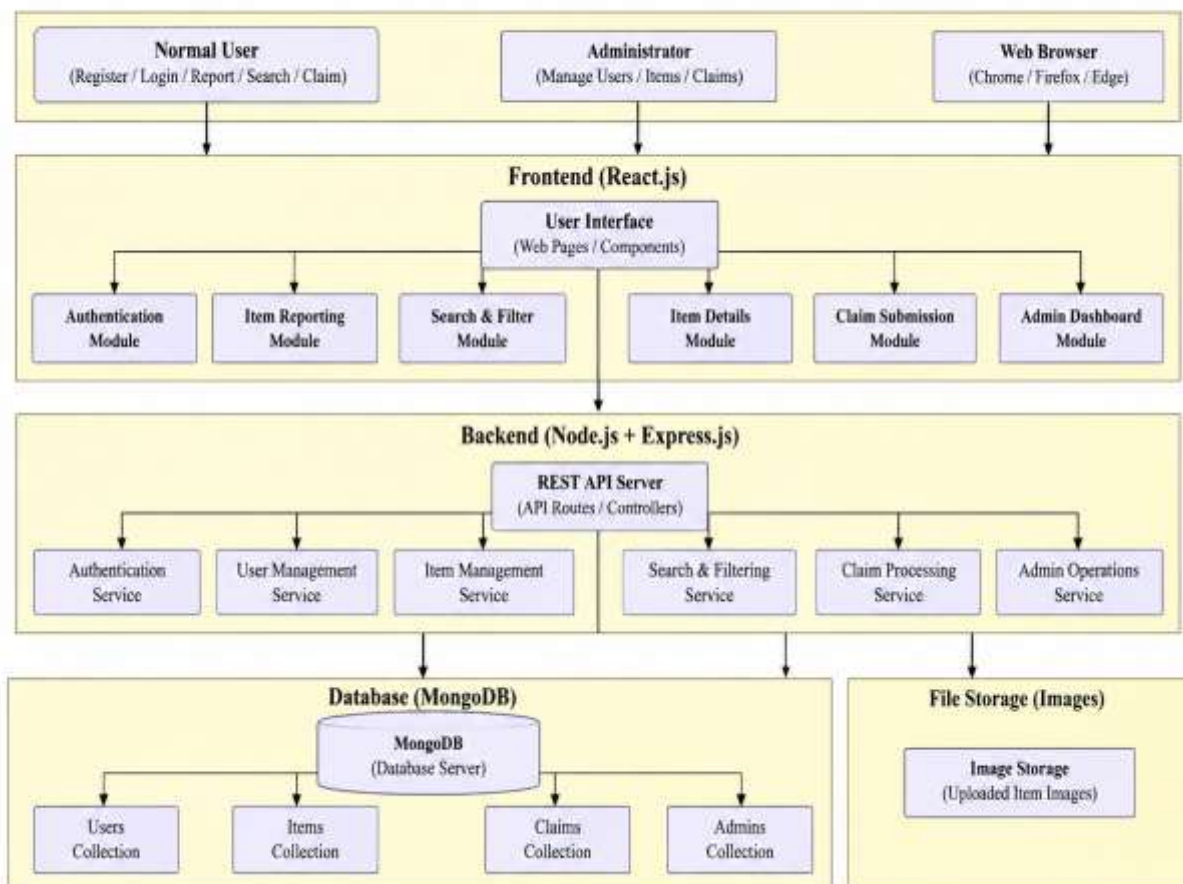
Visual Studio Code can be used for writing, organizing, debugging, and managing the project source code.

9. Web Browser

Modern browsers such as Google Chrome, Microsoft Edge, or Mozilla Firefox are required for accessing and testing the application.

PROJECT ARCHITECTURE

TECHNICAL ARCHITECTURE:



1. Frontend Layer

The frontend is developed using **React.js**. It provides the interface for user registration, login, item reporting, searching, viewing item details, and claim submission. Administrators access the system through a dedicated dashboard.

2. Backend Layer

The backend uses **Node.js and Express.js** to process requests from the frontend and manage the application's business logic. It handles:

- Authentication and Authorization
- User and Item Management
- Search and Filtering
- Claim Processing
- Admin Operations
- CRUD Operations
- REST API Responses

3. Database Layer

MongoDB is used to store application data in a flexible document-based structure. The main collections include:

- Users, Items, Claims, Admins

Overall Data Flow

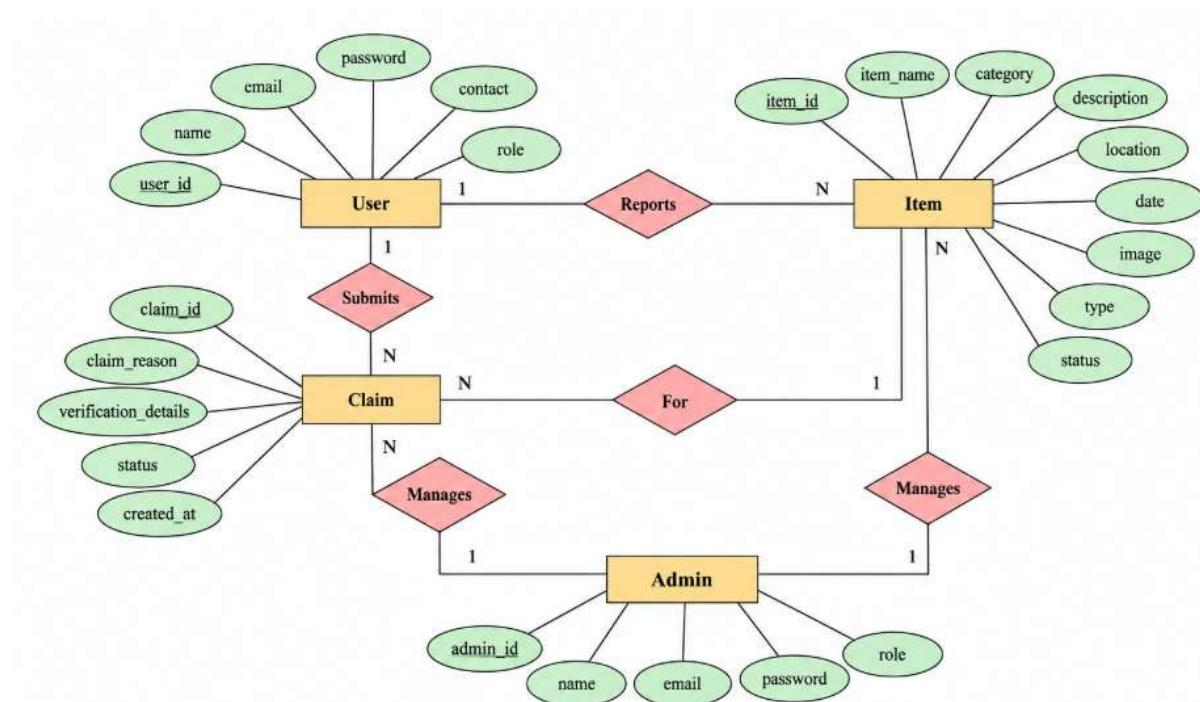
User → React.js Frontend → Node.js + Express.js API → MongoDB

The response follows:

MongoDB → Backend API → React.js Frontend → User

This architecture separates the **presentation, business logic, and data storage layers**, making the application easier to maintain and scale.

ER DIAGRAM



The Entity-Relationship (ER) diagram for **LostFound+** represents the major entities and their relationships within the Lost and Found Management System. The main entities include **User**, **Item**, **Claim**, and **Admin**. The User entity stores details such as user ID, name, email, password, contact information, and role.

The Item entity represents lost and found belongings and contains information such as item ID, item name, category, description, location, date, image, type, and status. The Claim entity stores ownership claim details including claim ID, claim reason, verification details, status, and creation date. The Admin entity contains administrator information such as admin ID, name, email, password, and role.

The relationships between these entities define how the system operates. A user can report multiple lost or found items, while each item is associated with the user who reported it. A user can submit ownership claims for reported items, and each claim is associated with a particular item. Administrators manage reported items and review ownership

KEY FEATURES:

LostFound+ provides several features to simplify lost and found management.

1. User Registration

Users can create an account by providing required information.

2. Secure Login

Registered users can log into the platform using their credentials.

3. Report Lost Item

Users can report belongings that they have lost by entering item information, location, date, description, and image.

4. Report Found Item

Users who find belongings can report them on the platform.

5. Search Items

Users can search for existing lost and found records using keywords.

6. Filtering

Items can be filtered based on relevant attributes such as category, item type, location, and status.

7. Item Details

Users can open an individual item and view its complete information.

8. Ownership Claim

A user can submit a claim when they identify an item that may belong to them.

9. Admin Verification

Administrators can review submitted claims before approving or rejecting them.

ROLES AND RESPONSIBILITIES:

USER:

The **User** is responsible for creating and maintaining an account on the LostFound+ platform by providing valid personal information and protecting their login credentials.

Users can report belongings that they have lost by entering details such as item name, category, description, location, date, and image. They can also report items that they have found so that the rightful owner can identify and recover them.

Users can search and browse available lost and found records using keywords and filters. They can view complete item information to identify possible matches and submit ownership claims by providing relevant information required for verification.

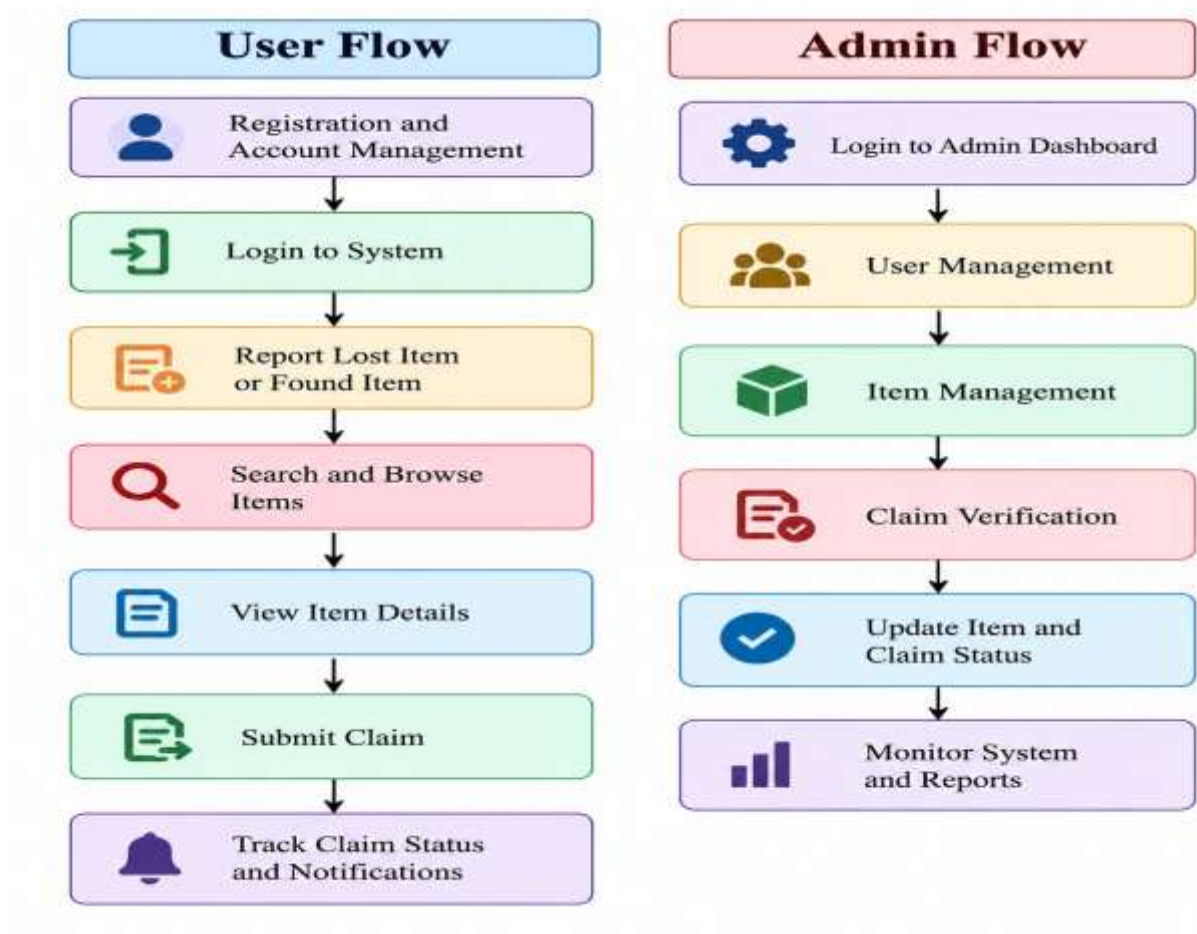
ADMIN:

The **Admin** is responsible for managing and monitoring the overall LostFound+ platform. The administrator can monitor registered users and manage user accounts when required. Administrators can review reported lost and found items and remove duplicate, invalid, or inappropriate listings.

They are also responsible for reviewing ownership claims and verifying the information submitted by users. After verification, the administrator can update the status of claims and reported items, such as pending, approved, rejected, or recovered. This helps maintain an organized, reliable, and secure LostFound+ system.

The **Admin** is responsible for managing the overall LostFound+ platform, monitoring registered users, reviewing reported lost and found items, and verifying ownership claims. maintains item and claim statuses to ensure that the recovery process is properly tracked.

USER FLOW & ADMIN FLOW



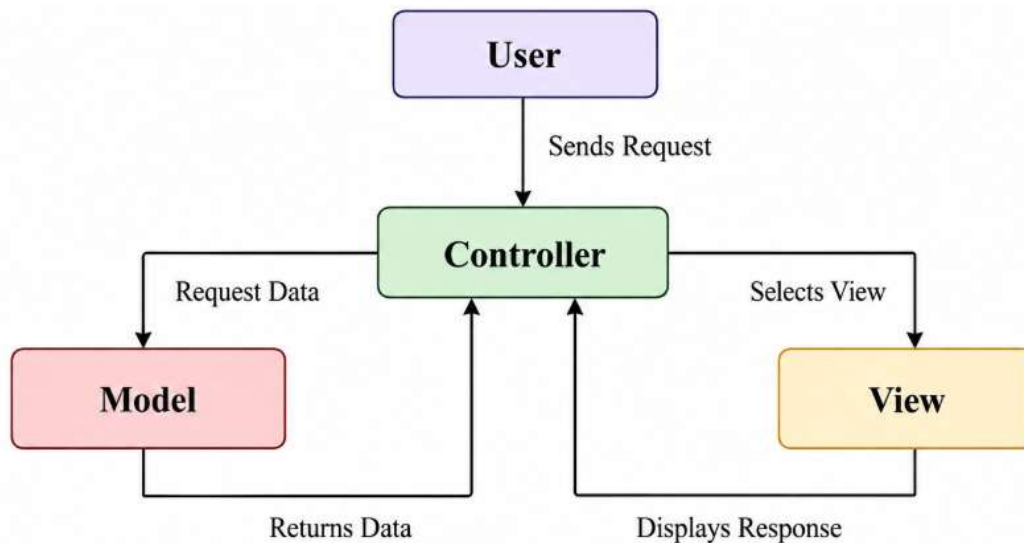
USER FLOW:

The user flow of LostFound+ describes the sequence of activities performed by a normal user. The user first creates an account and logs into the system. After login, the user can report a lost or found item by entering the required details. The user can then search and browse available items, view the details of a suitable item, and submit an ownership claim.

ADMIN FLOW:

The admin flow describes the activities performed by the administrator. The administrator first logs into the admin dashboard and manages the overall system. The admin can manage registered users, review reported lost and found items, and verify ownership claims submitted by users.

MVC PATTERN:



The **LostFound+** application follows an MVC-inspired architecture that separates the major responsibilities of the system and makes the application easier to develop and maintain.

The **Model** layer is responsible for managing the application's data and database operations. It represents the main entities of the system, including users, items, claims, and administrators. **MongoDB** is used to store and manage the application data.

The **Controller** layer handles the main business logic of the application. It processes user registration and login, manages lost and found items, performs search operations, handles claim submissions, and manages claim approval, rejection, and user-related operations.

The **Routes** layer defines the API endpoints through which the React frontend communicates with the backend. These routes handle authentication, item operations, claim operations, and administrator functions.

The **View** layer is represented by the **React.js frontend**, which provides the user interface of the application. It displays forms, dashboards, item cards, search results, item details, claim forms, and administrator panels.

The admin can manage registered users, review reported lost and found items, and verify ownership claims submitted by users.

This architecture provides **separation of responsibilities, better code organization, easier maintenance, reusable components, simpler debugging, and improved scalability**.

ADVANTAGES OF USING MVC IN THIS PROJECT:

The MVC architecture helps **LostFound+** maintain a clear separation between the application's data, business logic, and user interface. This makes the project easier to understand, develop, and maintain.

It improves **code organization** by keeping models, controllers, and routes in separate sections.

New features such as additional item categories, notification services, or claim-related functions can be added without affecting the entire application.

MVC also provides **reusability**, as controllers and models can be used by different parts of the system. Individual components can be tested and debugged more easily, which helps in identifying errors quickly.

The architecture also supports **scalability**, allowing LostFound+ to handle additional users, items, claims, and future modules as the application grows.

PROJECT SETUP AND CONFIGURATION

The React frontend can be created using Vite.

Example command:

```
npm create vite@latest client
```

Select:

React

JavaScript

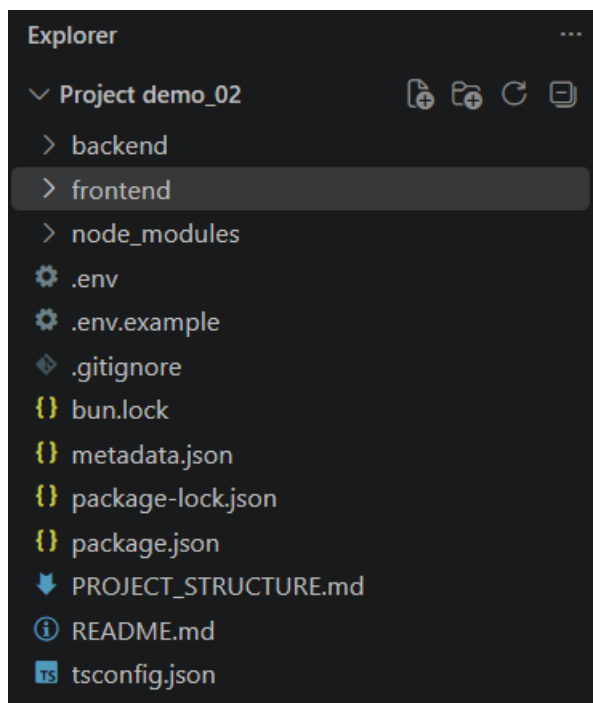
Install dependencies:

```
cd client
```

```
npm install
```

Run the frontend:

```
npm run dev
```



Frontend Setup

The React frontend can be created using Vite.

Example command:

```
npm create vite@latest client
```

Select:

React

JavaScript

Install dependencies

```
cd client
```

```
npm install
```

Run the frontend:

```
npm run dev
```

Backend Setup

Open the server directory:

```
cd server
```

Initialize Node.js:

```
npm init -y
```

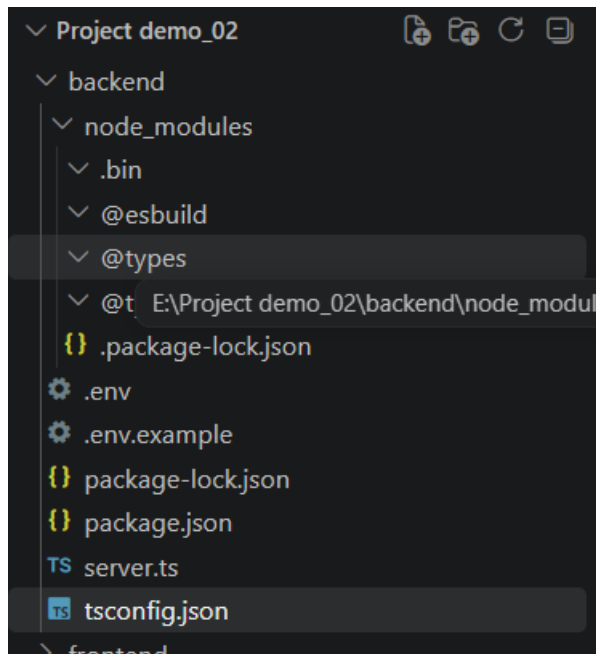
Install required backend packages such as:

```
npm install express mongoose cors dotenv
```

Additional packages can be installed depending on authentication, image upload, validation, and security requirements.

BACKEND DEVELOPMENT

Backend Structure:



authController.js

Handles registration and login-related operations.

itemController.js

Handles lost and found item operations.

claimController.js

Handles ownership claim submission and status management.

adminController.js

Handles administrator-specific operations.

authMiddleware.js

Checks whether a user is authenticated before allowing protected operations.

adminMiddleware.js

Ensures that only authorized administrators can access admin-specific functionality.

db.js

Contains the MongoDB connection configuration.

server.js

Acts as the main entry point of the backend application and initializes Express, middleware, routes, and database connectivity.

Item Management

The item controller handles operations such as:

- Add item, View items View individual item, Update item, Delete item, Search items , Filter items

Claim Management

The claim controller handles:

- Submit claim
- View claim
- Update claim
- Approve claim
- Reject claim

Admin Operations

Administrative APIs can provide functions for:

- User management
- Item moderation -Claim verification

DATABASE DEVELOPMENT:

MongoDB is used as the database management system for LostFound+.

MongoDB stores data as documents inside collections. This approach is suitable for the application because lost and found items can contain different types of information, including optional images and descriptive fields.

```
import express, { Request, Response, NextFunction } from 'express';
import path from 'path';
import { fileURLToPath } from 'url';
import jwt from 'jsonwebtoken';
import bcrypt from 'bcryptjs';
import dotenv from 'dotenv';
import cors from 'cors';

dotenv.config({ path: path.resolve(process.cwd(), '.env') });
dotenv.config();

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const PORT = parseInt(process.env.PORT || '5000', 10);
const JWT_SECRET = process.env.JWT_SECRET || 'lostfound-plus-jwt-secret-key-2026';
const FRONTEND_URL = process.env.FRONTEND_URL || 'http://localhost:5173';

app.use(cors({ origin: [FRONTEND_URL, 'http://localhost:5173', 'http://localhost:3000'], credentials: true }));
app.use(express.json());

// Health check for frontend / deploy probes
app.get('/api/health', (_req: Request, res: Response) => {
  res.json({ status: 'ok', service: 'lostfound-backend' });
});
```

Main Collections

1. Users

Stores registered user information.

2. Items

Stores lost and found item information.

3. Claims

Stores ownership claim information.

4. Admins

Stores administrator information or role-based administrative records.

Database Connection

The backend establishes a connection with MongoDB when the application starts.

Environment variables can be used for sensitive configuration such as database connection strings.

Example:

`MONGODB_URI=your_database_connection`

Data Operations

The application performs:

- Create
- Read
- Update
- Delete

operations on database records.

Data Integrity

Validation should be applied to important fields before storing data.

- Required item name
- Valid category
- Valid date
- Valid user reference
- Valid claim information

This helps maintain consistent and reliable records.

package-lock.json

```
1 {
2   "name": "lostfound-backend",
3   "version": "0.0.0",
4   "lockfileVersion": 3,
5   "requires": true,
6   "packages": {
7     "node_modules/@esbuild/win32-x64": {
8       "version": "0.28.2",
9       "resolved": "https://registry.npmjs.org/@esbuild/win32-x64/-/win32-x64-0.28.2.tgz",
10      "integrity": "sha512-5ebpx3k@zrl/rnU17553kuee0bHL/Gq0WTF9|vcpv73wAp5eu8MF8UpK9bh4vZjj7yX8etf/8tI8Ney695g--",
11      "cpu": [
12        "x64"
13      ],
14      "dev": true,
15      "license": "MIT",
16      "optional": true,
17      "os": [
18        "win32"
19      ],
20      "engines": {
21        "node": ">=18"
22      }
23     },
24     "node_modules/@types/bcryptjs": {
25       "version": "3.0.0",
26       "resolved": "https://registry.npmjs.org/@types/bcryptjs/-/bcryptjs-3.0.0.tgz",
27       "integrity": "sha512-MR2OuCuaz8UC2ZE4R5H1co2goQ512XcJ6Y3hbM/xDvmmqf4d4|voolmsMx650K6CTNkbn25YL+YuuuK7c1dg==",
28       "deprecated": "This is a stub types definition. bcryptjs provides its own type definitions, so you do not need this installed.",
29       "dev": true,
30       "license": "MIT",
31       "dependencies": {

```

Package.json

```
> package.json > ...
{
  "name": "lostfound-backend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  > Debug
  "scripts": {
    "dev": "tsx server.ts",
    "start": "tsx server.ts",
    "lint": "tsc --noEmit"
  },
  "dependencies": {
    "@google/genai": "^2.24.0",
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.5",
    "dotenv": "^17.2.3",
    "express": "^4.21.2",
    "jsonwebtoken": "^9.0.3"
  },
}
```

Server.js

```
import express, { Request, Response, NextFunction } from 'express';
import path from 'path';
import { fileURLToPath } from 'url';
import jwt from 'jsonwebtoken';
import bcrypt from 'bcryptjs';
import dotenv from 'dotenv';
import cors from 'cors';

dotenv.config({ path: path.resolve(process.cwd(), '.env') });
dotenv.config();

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const PORT = parseInt(process.env.PORT || '5000', 10);
const JWT_SECRET = process.env.JWT_SECRET || 'lostfound-plus-jwt-secret-key-2026';
const FRONTEND_URL = process.env.FRONTEND_URL || 'http://localhost:5173';

app.use(cors({ origin: [FRONTEND_URL, 'http://localhost:5173', 'http://localhost:3000'], credentials: true }));
app.use(express.json());

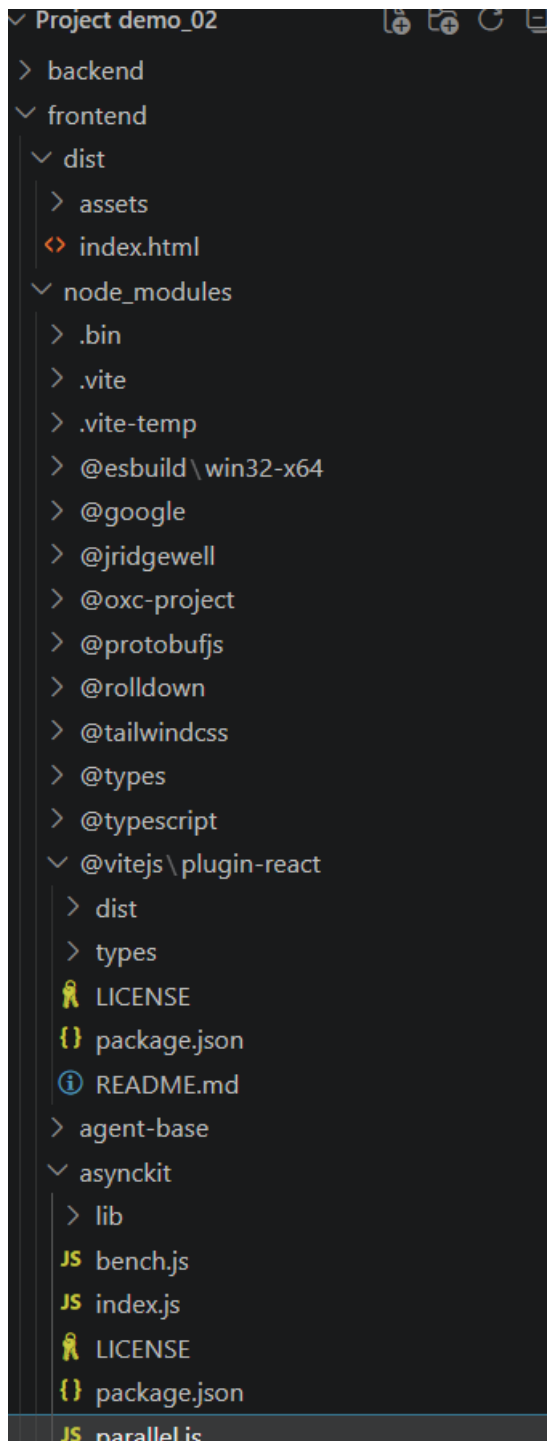
// Health check for frontend / deploy probes
app.get('/api/health', (_req: Request, res: Response) => {
  res.json({ status: 'ok', service: 'lostfound-backend' });
});
```

Tsconfig.json

```
1  {
2    "compilerOptions": {
3      "target": "ES2022",
4      "module": "ESNext",
5      "moduleResolution": "bundler",
6      "lib": ["ES2022"],
7      "skipLibCheck": true,
8      "isolatedModules": true,
9      "allowImportingTsExtensions": true,
10     "noEmit": true,
11     "types": ["node"]
12   },
13   "include": ["server.ts"]
14 }
15
```

FRONT-END DEVELOPMENT:

Frontend Structure:



The frontend of LostFound+ is developed using React.js.

React provides a component-based architecture that allows the interface to be divided into reusable components.

USER INTERFACE:

The LostFound+ user interface is designed to provide a simple and organized experience.

Landing Page

The landing page introduces the LostFound+ platform and provides options for registration and login.

Registration

New users can create an account by entering their basic details.

Login

Existing users can securely access their accounts.

Dashboard

- Report Lost Item
- Report Found Item
- Search Items
- My Reports
- My Claims

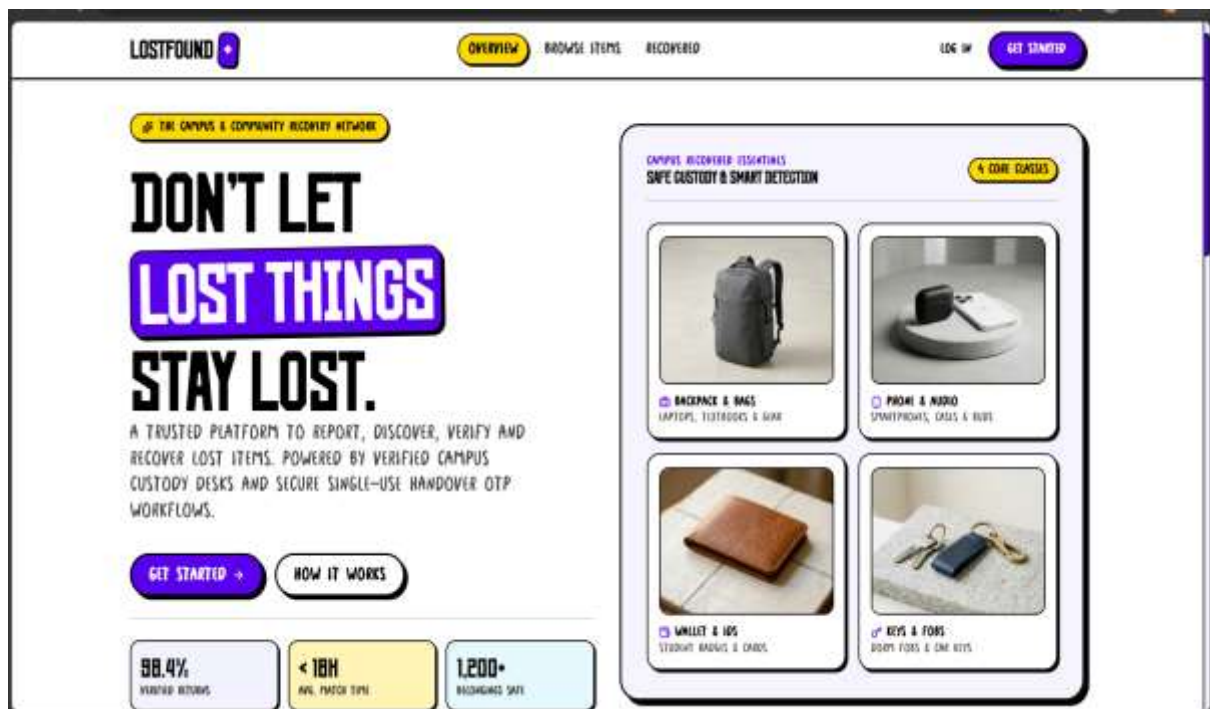
ADMIN DASHBOARD:

The LostFound+ administrator dashboard provides centralized control over the platform.

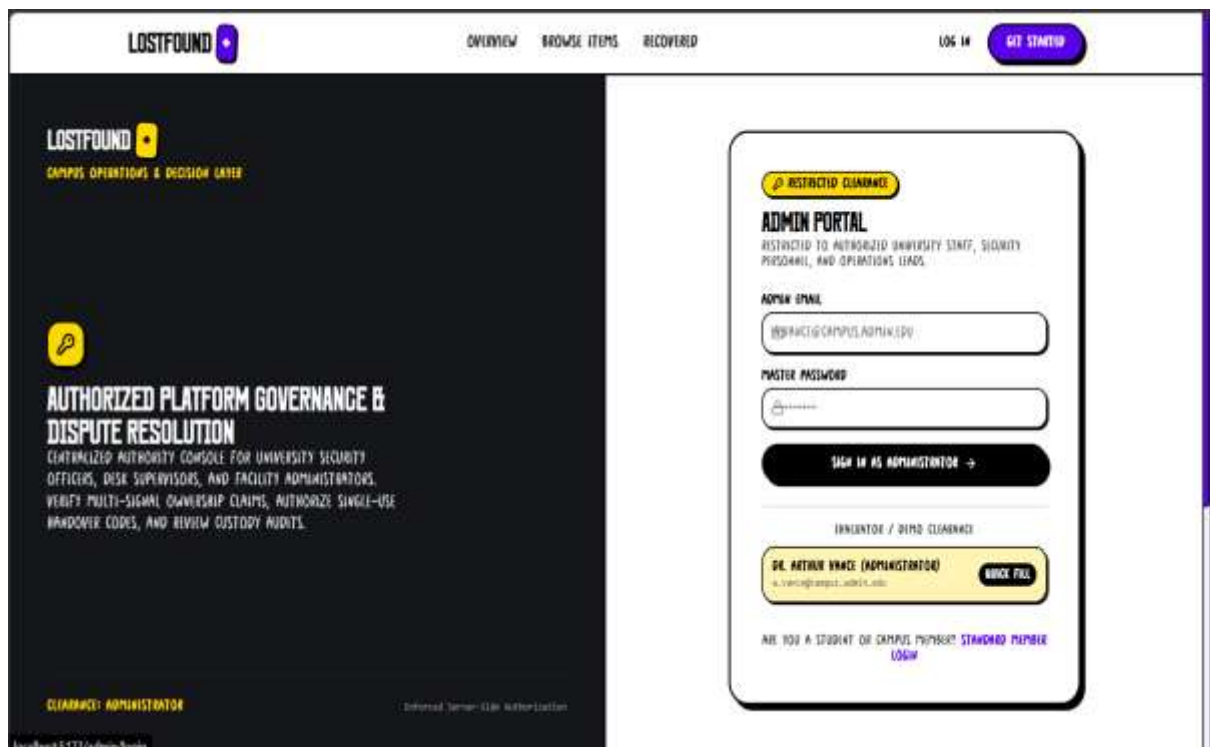
The dashboard can display summary information such as:

- Total Users
- Total Lost Items
- Total Found Items
- Pending Claims
- Approved Claims
- Rejected Claims

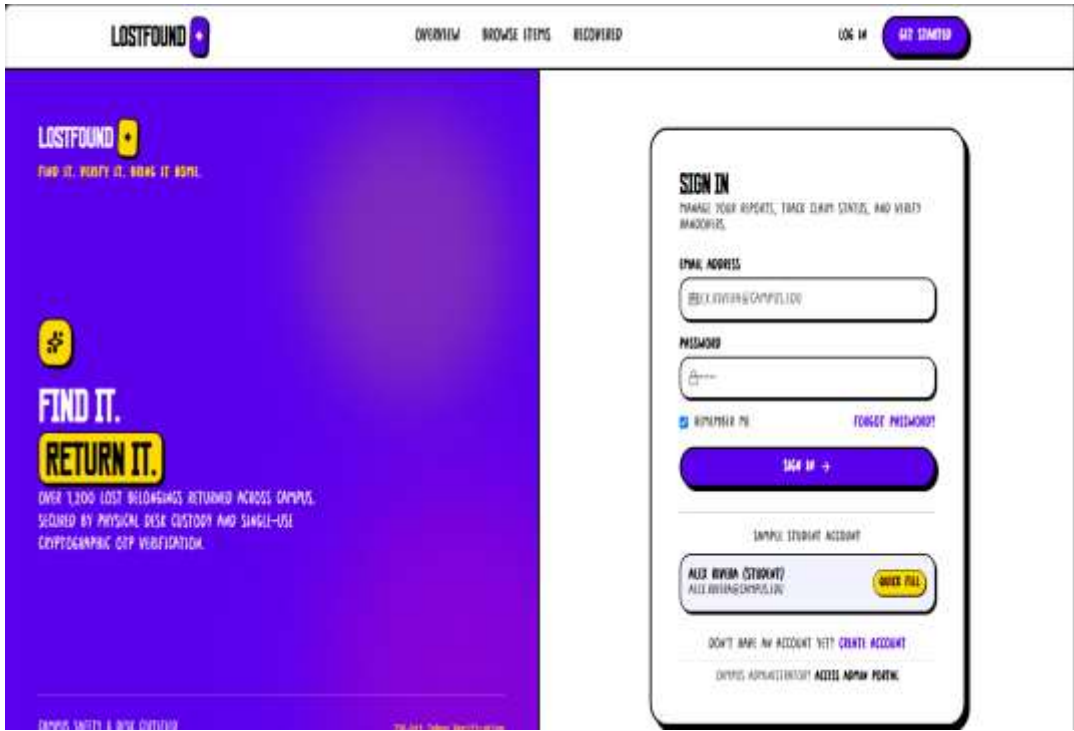
Landing Page:



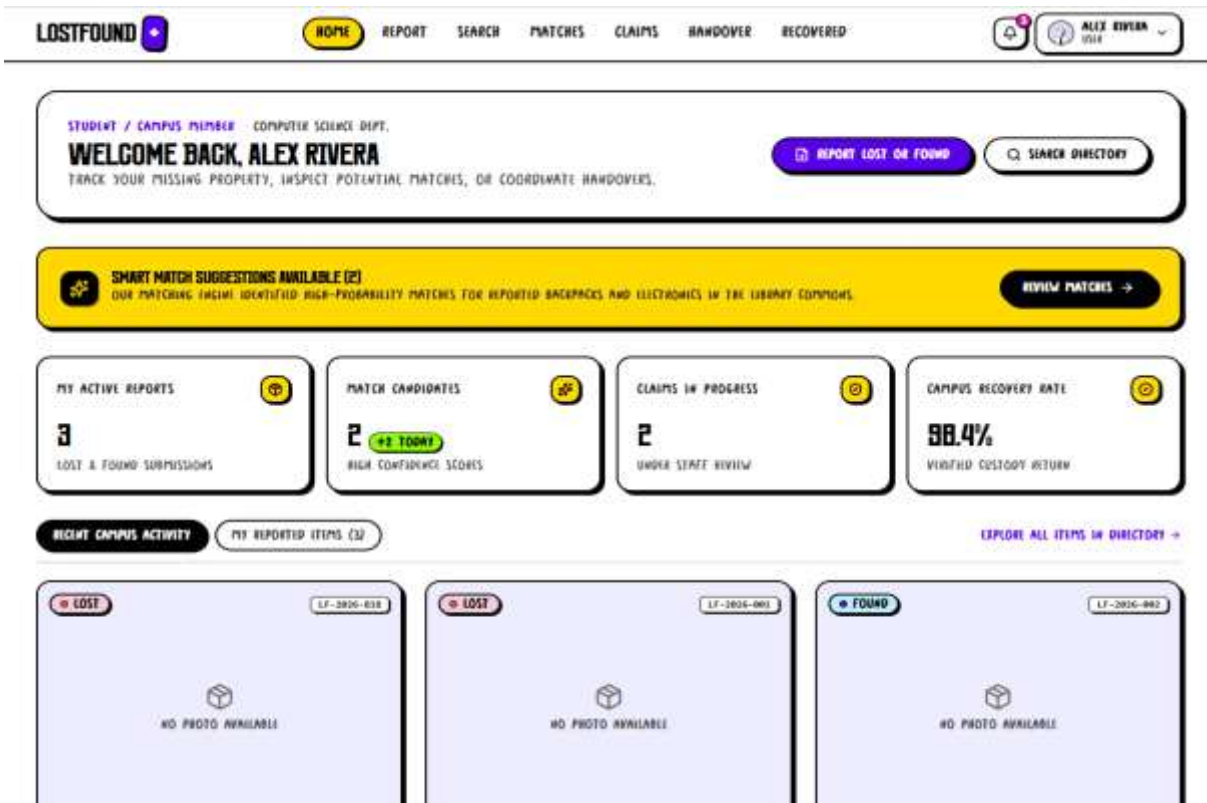
Admin Login Page:



User Login Page:



Dashbord Page:



Report Page:

LOSTFOUND

HOMEREPORTSEARCHMATCHESCLAIMSHandoverRecovered

ALEX RIVERA

USER

1LOCATION & TIMESTAMP

CAMPUS LOCATION / LANDMARK *
MAIN CAMPUS LIBRARY - 2ND FLOOR STUDY COMMONS

DATE LOST
30-07-2028

APPROXIMATE TIME
02:00 PM

2PHOTO & OWNERSHIP VERIFICATION QUESTION

UPLOAD PHOTO (OPTIONAL, OR SELECT SHINY ASSET)
CLICK TO UPLOAD OR CHOOSE A SAMPLE ASSET BELOW
BACKPACKENRINGSWALLETBAGS

☐ CONFIDENTIAL VERIFICATION QUESTION
ONLY THE SCHOOL COUNCILORS WILL KNOW THIS ANSWER, USED BY VERIFICATION OFFICERS TO PREVENT FRAUDULENT CLAIMS.

VERIFICATION QUESTION
E.G. WHAT COLOR IS THE INNER NOTEBOOK COVER?

EXPECTED ANSWER (HIDDEN FROM PUBLIC)
E.G. NAVY BLUE WITH PHYSICS FORMULAS

3REPORTER CONTACT INFORMATION

YOUR NAME
ALEX RIVERA

EMAIL ADDRESS
ALEX.RIVERA@CAMPUS.EDU

MOBILE PHONE
+1 (555) 234-5678

SUBMISSIONS ARE RECORDED IN THE CAMPUS CRIME & EVIDENCE LOG.

CANCEL

SUBMIT REPORT

Search Page:

LOSTFOUND

HOMEREPORTSEARCHMATCHESCLAIMSHandoverRecovered

ALEX RIVERA

USER

FIND MY ITEM (SMART MATCHES)

PUBLIC DIRECTORY (3)

Multi-Signal Correlation Engine

MULTI-SIGNAL RECOVERY ENGINE: FIND IT. VERIFY IT. BRING IT HOME.

FIND MY ITEM

SEARCH ACROSS CAMPUS RECOVERED ITEMS USING YOUR PHOTO, A WRITTEN DESCRIPTION, OR COMBINE BOTH FOR HIGHER CONFIDENCE.

YOU DON'T NEED A PHOTO TO SEARCH.

A PHOTO MAKES MATCHING EASIER, BUT YOUR DESCRIPTION, CAMPUS LOCATION, DATE, AND DISTINCT MARKS ARE COMPLETELY ENOUGH TO BEGIN RECOVERY.

HOW WOULD YOU LIKE TO FIND YOUR ITEM?

CHOOSE THE SEARCH METHOD THAT BEST FITS WHAT INFORMATION OR MATERIALS YOU HAVE RIGHT NOW.

VISUAL MATCHING

SEARCH WITH AN IMAGE

UPLOAD A PHOTO OF THE ITEM AND COMPARE IT WITH RECOVERED ITEM IMAGES.

BEST IF YOU HAVE A PHOTO OF YOUR ITEM OR RECEIPT.

NO PHOTO NEEDED

SEARCH WITH DETAILS

DESCRIBE THE ITEM, WHERE YOU LOST IT, WHEN YOU LOST IT, AND OTHER IDENTIFYING DETAILS.

MOST COMMON, 100% PHOTO-FREE SEARCH.

HIGHEST ACCURACY

USE BOTH

COMBINE VISUAL AND DESCRIPTIVE EVIDENCE FOR A STRONGER MATCH.

FUSES MULTI-MODAL INPUT & TEXT ANALYSIS.

Find My Items Page:

LOSTFOUND

HOMEREPORTSEARCHMATCHESCLAIMSMANDOVERRECOVERED

🔔

🕒

ALEX RIVERA

USER

FIND MY ITEM (SMART MATCHES)

PUBLIC DIRECTORY (3)

MULTI-SIGNAL Correlation Engine

MULTI-SIGNAL RECOVERY ENGINE: FIND IT. VERIFY IT. BRING IT HOME.

FIND MY ITEM

SEARCH ACROSS CAMPUS RECOVERED ITEMS USING YOUR PHOTO, A WRITTEN DESCRIPTION, OR COMBINE BOTH FOR HIGHER CONFIDENCE.

CHANGE SEARCH METHOD

SEARCH CONFIGURATIONMETHOD 1: VISUAL IMAGE COMPARISON

SWITCH METHOD

📁 UPLOAD YOUR ITEM IMAGE

ITEM, FILE, UPLOAD

📁

DROP & DROP OR CHOOSE IMAGE

CLAR PHOTOS SHOWING COLOR, SHAPE, AND BRANDS WORK BEST.

OR SELECT SAMPLE ITEM TO TEST:

BACKPACK

LAPTOP

WALLET

KEYS

ACTIVE IMAGE PREVIEW



IMAGE LOADED & READY FOR COMPARISON

SEARCH TRIGGERS MULTI-SIGNAL CANDIDATE BRANDING WITHOUT PUBLISHING PUBLIC POSTS.

FIND MATCHING ITEMS →

Search Page:

MULTI-SIGNAL RECOVERY ENGINE: FIND IT. VERIFY IT. BRING IT HOME.

FIND MY ITEM

SEARCH ACROSS CAMPUS RECOVERED ITEMS USING YOUR PHOTO, A WRITTEN DESCRIPTION, OR COMBINE BOTH FOR HIGHER CONFIDENCE.

CHANGE SEARCH METHOD

SEARCH CONFIGURATIONMETHOD 2: DESCRIPTION & SPECIFICATIONS (YOUR PHOTO-FREE)

SWITCH METHOD

📁 ITEMS DESCRIPTION & LOCATION EVIDENCE

ITEM TITLE

BLACK BACKPACK

CATEGORY

BAGS & BACKPACKS

COLOR

BLACK

BRAND / MANUFACTURER

THE NORTH FACE

LOCATION LOST

MAIN CAMPUS LIBRARY - 2ND FLOOR STUDY COMMONS

DATE LOST

25-05-2026

APPROXIMATE TIME

02:30 PM

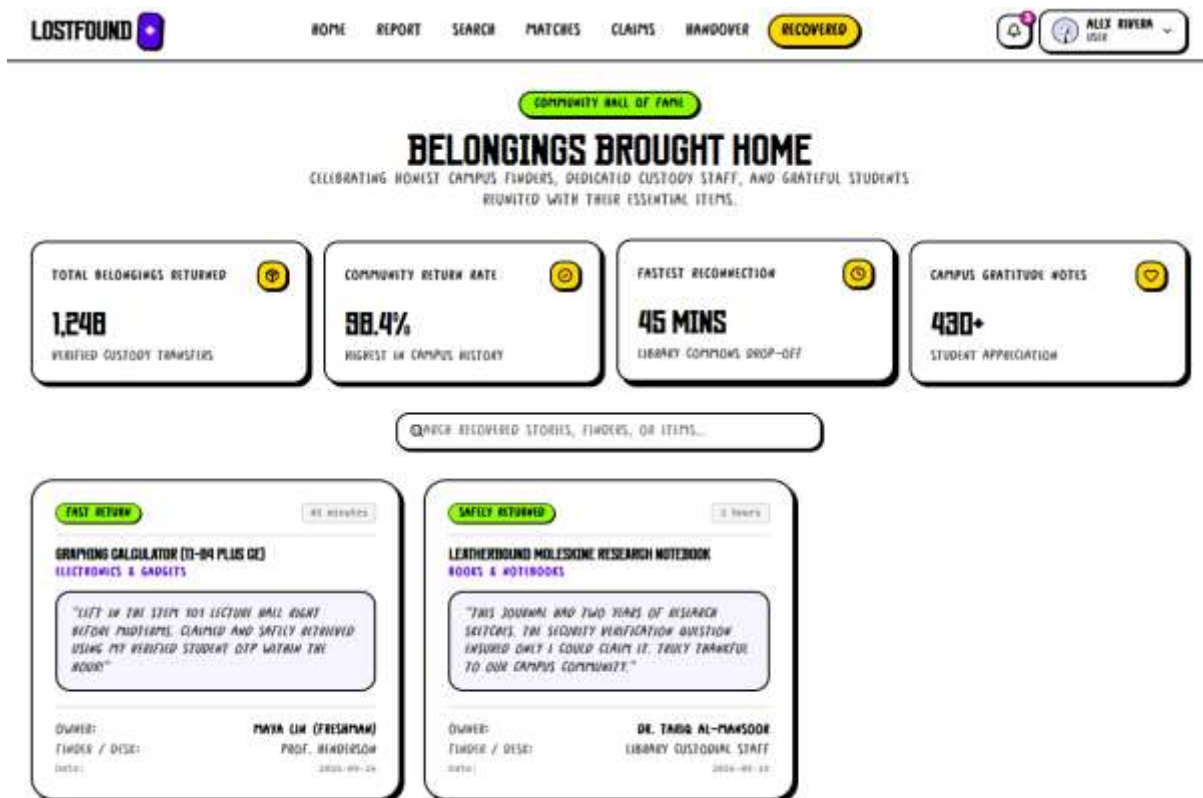
DESCRIPTION

BLACK COMPUTER BACKPACK WITH BLUE BOTTLE AND COLLEGE NOTEBOOK INSIDE.

DISTINCTIVE FEATURES

SMALL REFLECTIVE TAB AND STICKER ON THE SIDE POCKET.

Found page:



Demo Video Link:

<https://drive.google.com/file/d/1dHSuNwk2-kGEWbYzRNZ8CgaMQq7N-2-U/view?usp=sharing>

Code Drive Link:

https://drive.google.com/file/d/1MsZU6i0JgAFyBndT_kqcVfzAV0WfRdxS/view?usp=sharing

Github link:

https://github.com/Akilesh-techie47/Team_Heisenbugs